PionCloud

Pion Cloud Model Structure Function Calculator

C++ port of the Hobbs–Melnitchouk Fortran pion-cloud suite

User Manual — Version 1.3

Abstract

PionCloud is a modular C++ library and executable for computing the leading pion-cloud contributions to nucleon structure functions $F_2^{(\pi N)}(x)$ and $F_2^{(\pi N)}(\theta_h)$ in the Sullivan process. It is a direct, validated port of three Fortran programs (`3Var_x.f`, `3Var_theta.f`, `f2_pi_sub.f90`) by T. Hobbs and W. Melnitchouk, extended with configurable scan modes, ROOT output, CTEQ6 neutron structure function evaluation, and a static library interface for embedding in other programs. The code was developed as a framework for BoNuS/TDIS analysis at JLab.

Last updated: November 2026

Contents

1 Introduction **3**
1.1 Physics background . . . 3
1.2 Code history . . . 3

2 Building **3**
2.1 Prerequisites . . . 3
2.2 Full build (ROOT + CTEQ6) . . . 4
2.3 Without ROOT or CTEQ6 . . . 4
2.4 Direct g++ (no CMake) . . . 4
2.5 CTEQ6 table files . . . 4

3 Running **4**
3.1 Quick start . . . 4
3.2 Scan modes . . . 5
3.2.1 `x` scan . . . 5
3.2.2 `theta` scan . . . 5
3.2.3 `kbin` scan . . . 5

4 Input file reference **5**
4.1 Beam kinematics . . . 6
4.2 Physics — channel selection . . . 6
4.3 Physics — form factor . . . 6
4.4 Physics — cutoff parameters . . . 6
4.5 Physics — nucleon and PDF version . . . 7
4.6 Run control and usability . . . 7
4.7 x-scan settings . . . 7
4.8 theta-scan settings . . . 7
4.9 kbin-scan settings . . . 8
4.10 Output settings . . . 8

5 Output files **8**
5.1 ROOT objects . . . 8

6 What's new in v1.3 **9**
6.1 Nucleon selector . . . 9
6.2 GRV99 pion PDF . . . 9
6.3 Precision preset . . . 9
6.4 Inclusive mode shortcut . . . 9
6.5 Verbosity control . . . 10
6.6 Angular cut in degrees . . . 10
6.7 CTEQ6 table directory . . . 10
6.8 Single-point eval() for generator use . . . 10
6.9 C interface for Python/Fortran . . . 10

7 Worked examples: the published TDIS figures **11**
7.1 Fig. 47: angular distribution (theta scan) . . . 11
7.2 Fig. 46: x dependence, full acceptance (x scan) . . . 12
7.3 Fig. 5: proton target, four $|\vec{k}|$ bins (kbin scan) . . . 12
7.4 Fig. 6: neutron, πN, ρN, $\pi\Delta$ channels (kbin scan) . . . 13
7.5 Fig. 37: neutron, $\Delta|\vec{k}| = [250, 400]$ MeV (kbin scan) . . . 15
7.6 Summary of the comparison . . . 16

8 Class and API reference **16**
8.1 RunParams (PhysicsParams.hh) 16
8.2 Calculator (Calculator.hh) 17
8.3 ScanResults and data structures (Results.hh) 17
8.4 InputReader (InputReader.hh) 18
8.5 OutputWriter (OutputWriter.hh) 18
8.6 CTEQ6 (CTEQ6.hh) 18
8.7 Physics functions (SplittingFunctions.hh, GRV.hh, Kinematics.hh) 19

9 Using PionCloud as a library **19**

10 CLI reference **20**

11 Project layout **20**

12 Known issues and TODO **21**

Introduction

Physics background

In the Sullivan process, a nucleon N fluctuates into a virtual meson–baryon pair (MB):

$$N \longrightarrow M + B \tag{1}$$

The leading contributions from the pion cloud are:

$$n \to \pi^- + p \quad \text{(charge exchange, } dis = 0) \tag{2}$$
$$p \to \pi^0 + p \quad \text{(neutral exchange, } dis = 1) \tag{3}$$

with subdominant contributions from ρN and $\pi\Delta$ channels.

The tagged structure function, observed when the recoil proton is detected in a specific momentum and angular window, is:

$$F_2^{(\pi N)}(x;\, \Delta k,\, \Delta\theta_h) = \int_{\Delta k,\, \Delta\theta_h} \rho_{\mathrm{PS}}(\theta_h, k)\; f_{\pi N}(y, k_T)\; F_2^{\pi}(x/y,\, Q^2)\; dk\, d\theta_h \tag{4}$$

where ρ_{PS} is the phase-space Jacobian, $f_{\pi N}$ is the pion splitting function, and F_2^{π} is the pion structure function evaluated using GRV92 parton distributions.

The ratio $F_2^{(\pi N)}/F_2^{n}$ is the core BoNuS observable — it gives the fractional pion-cloud contribution to the neutron structure function for a given experimental acceptance.

Code history

The original Fortran programs were written by T. Hobbs (2013–2014) with modifications by W. Melnitchouk. The C++ port (PionCloud) preserves the physics exactly while adding configurability, a library interface, and ROOT output.

Key references:

- TDIS proposal [1], Sec. “Pion cloud model” and Figs. 5, 6, 37, 46, 47: the calculations that the Fortran codes were written for and that PionCloud reproduces (Sec. 7). Also T. J. Hobbs [3], Fig. 2.
- Holtmann, Szczurek, Speth (HSS) [4]: Λ values constrained by inclusive $pp \to nX$ data, used as defaults (labelled “HSS” in the Fortran sources).
- Glück, Reya, Vogt (GRV92) [5]: pion parton distributions.
- Pumplin et al. (CTEQ6) [6]: nucleon PDFs.

Building

Prerequisites

- CMake ≥ 3.14 or g++ ≥ 7
- C++17 or later
- **gfortran** (for CTEQ6 Fortran library; optional but recommended)
- **ROOT** ≥ 6 (optional; required for `.root` output)

Full build (ROOT + CTEQ6)

```
source /path/to/root/bin/thisroot.sh
cmake -B build -DUSE_ROOT=ON -DUSE_CTEQ6=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
# binary: build/PionCloud
# tests:  build/PionCloudTests
```

Without ROOT or CTEQ6

```
cmake -B build -DUSE_ROOT=OFF -DUSE_CTEQ6=OFF
cmake --build build
```

Without CTEQ6: F_2^n and the ratio are set to zero. Without ROOT: only ASCII output is produced.

Direct g++ (no CMake)

```
# No optional dependencies
g++ -std=c++17 -O3 -Iinclude \
    src/Calculator.cc src/InputReader.cc src/OutputWriter.cc src/main.cc \
    -o PionCloud

# With ROOT and CTEQ6
gfortran -O2 -c CTEQ6/Cteq6Pdf-2007.f -o Cteq6Pdf.o
g++ -std=c++17 -O3 -Iinclude -DHAVE_ROOT -DHAVE_CTEQ6 \
    src/Calculator.cc src/InputReader.cc src/OutputWriter.cc src/main.cc \
    Cteq6Pdf.o $(root-config --cflags --libs) -lgfortran -o PionCloud
```

CTEQ6 table files

The CTEQ6 library reads grid tables from the *current working directory* at runtime. Place the following files in `CTEQ6/` (CMake copies them to the build directory automatically):

Iset	Name	File
1	CTEQ6M (NLO MSbar, default)	`cteq6m.tbl`
2	CTEQ6D (NLO DIS)	`cteq6d.tbl`
3	CTEQ6L (LO)	`cteq6l.tbl`
4	CTEQ6L1 (LO, α_s fitted)	`cteq6l1.tbl`

Running

Quick start

```
# 1. Generate a documented template input file
./PionCloud --write-template myrun.in

# 2. Edit the template, then run
./PionCloud --input myrun.in

# 3. Override any parameter from the command line
./PionCloud --input myrun.in --kmin 0.100 --kmax 0.200 --out Run_100_200
```

Scan modes

PionCloud has four scan modes, selected by **scan_mode**:

Mode	Output	Integrated over	Mirrors
x	$F_2^{(\pi N)}(x)$	$(\theta_h, \lvert\vec{k}\rvert)$	3Var_x.f
theta	$F_2^{(\pi N)}(\theta_h)$	$(x, \lvert\vec{k}\rvert)$	3Var_theta.f
kbin	$F_2^{(\pi N)}(x; \Delta k, \Delta\theta_h)$	(y, k_T) with $\lvert\vec{k}\rvert$ and $\cos\phi_k$ gates	f2_pi_sub.f90
all	all of the above	—	—

3.2.1 x scan

Integrates over the hadron production angle θ_h and the recoil proton momentum magnitude $|\vec{k}|$ at each output Bjorken-x point. Matches 3Var_x.f exactly. Useful for computing $F_2^{(\pi N)}(x)$ as a function of x for a given $|\vec{k}|$ tagging window.

3.2.2 theta scan

Integrates over x and $|\vec{k}|$ at each output angle θ_h. Matches 3Var_theta.f exactly. Used to produce plots of the angular distribution $F_2^{(\pi N)}(\theta_h; \Delta x, \Delta k)$ such as Fig. 47 of the TDIS proposal [1] (reproduced in Fig. 1). The internal x and $|\vec{k}|$ integration ranges are controlled by **x_min_th**, **x_max_th**, **k_min_th**, **k_max_th**:

- **Left panel** (Fig. 47): use defaults ($\Delta x = [0, 0.6]$, $\Delta k = [0, 500]$ MeV).
- **Right panel** (Fig. 47): set **x_min_th=0.05**, **k_min_th=0.060**, **k_max_th=0.250**.

3.2.3 kbin scan

Mirrors the f2_pi_sub.f90 subroutine. Integrates over (y, k_T) at each output x point with:

1. An angular acceptance gate on $\cos\phi_k \in$ [cosph_min, cosph_max] (default: $\cos 70°$–$\cos 30°$, i.e. [0.342, 0.866]).
2. A $|\vec{k}|$ magnitude window [k_min, k_max].

The $|\vec{k}|$ magnitude is computed analytically from (k_T, y):

$$|\vec{k}| = \sqrt{k_T^2 + \frac{\left[k_T^2 + (1-(1-y)^2)m_N^2\right]^2}{4\,m_N^2\,(1-y)^2}} \tag{5}$$

This mode is used to produce the four $|\vec{k}|$-bin curves and the individual channel contributions (πN, ρN, $\pi\Delta$) of Figs. 5, 6 and 37 of Ref. [1] (Figs. 3–5).

Note that $\cos\phi_k > 0$ for every (y, k_T), so the recoil nucleon always goes forward and an upper angle of 90° is the same as 180°. For $x \geq 0.05$ and $|\vec{k}| \leq 400$ MeV the 70° upper cut is also practically inactive (effect $< 0.5\%$), because $y \geq x$ keeps the recoil forward of $\approx 70°$.

Input file reference

The input file uses a simple INI-style format. Lines beginning with # are comments. Section headers ([...]) are accepted but ignored. All keys are case-insensitive.

Beam kinematics

Key	Default	Description
beam_energy	11.0	Electron beam energy E [GeV]
electron_angle	35.0	Electron scattering angle θ_e [deg]

Note: `3Var_theta.f` and `3Var_x.f` hardcode $\theta_e = 35°$; θ_e only enters through $Q^2(x)$. The published TDIS figures [1] are reproduced with $\theta_e = 35°$ for *all* modes, including kbin (Sec. 7). The value $\theta_e = 12°$ used when `f2_pi_sub` was called from `f2_pi_ranges.c` gives a lower Q^2 and tagged curves that are 3–10% higher at $x \gtrsim 0.15$.

Physics — channel selection

Key	Default	Description
contribution	`pion`	Meson-baryon channel: `pion` (0), `rho` (1), `pi_delta` (2)
dissociation	`charge_exchange`	Charge config: `charge_exchange` (0), `neutral` (1)

Charge exchange (*dis*=0): $n \to \pi^- + p$ (or $p \to \pi^+ n$). Isospin factor = 2. Used for the πN, ρN, $\pi\Delta$ comparisons (Figs. 6 and 37 of [1]).

Neutral (*dis*=1): $p \to \pi^0 + p$. Isospin factor = 1. Used for the proton-target curves (Figs. 5, 46, 47 of [1]).

Each run gives one charge state. The total pion-cloud contribution of a nucleon sums both charge states (isospin factors $1 + 2 = 3$). Multiply a *neutral* result by 3, or a *charge-exchange* result by 3/2, to obtain the total $F_2^{(\pi N)}(x)$ (the violet "fully integrated" curves of the published figures; see Sec. 7).

Physics — form factor

Code	Name	Notes
0	`monopole`	$(L^2 + m_N^2)/(L^2 + s_{\pi N})$
1	`dipole`	monopole squared
2	`exp_s`	$\exp[(m_N^2 - s_{\pi N})/L^2]$; HSS default
3	`cov_dipole`	covariant dipole in t
4	`dipole_sch`	dipole, s-channel Λ exchange
5	`exp_t`	$\exp[(t - m_\pi^2)/L^2]$
6	`pauli_villar`	Pauli–Villar; **C++ default**

Physics — cutoff parameters

Key	Default	Description
lambda	1.33	Λ for $\pi N/\rho N$ vertex [GeV]
lambda_delta	1.39	Λ for $\pi\Delta$ vertex [GeV]

For the s-dependent exponential form factor (`exp_s`), $G_{\pi N} = \exp[(M^2 - s_{\pi N})/\Lambda^2]$, the values constrained by inclusive $pp \to nX$ data [4] (converted to this convention, see [1]) are $\Lambda_{\pi N} = \Lambda_{\rho N} = 1.56 \pm 0.07$ GeV and $\Lambda_{\pi\Delta} = 1.39 \pm 0.07$ GeV. The $\pi\Delta$ channel always uses **lambda_delta**; **lambda** has no effect on it. Recommended values for $\Lambda_{\pi N}$:

Value	Meaning
$\Lambda = 1.56$ GeV	HSS central value
$\Lambda = 1.63$ GeV	HSS $+1\sigma$
$\Lambda = 1.48$ GeV	HSS -1σ

Physics — nucleon and PDF version

Key	Default	Description
nucleon	neutron	neutron or proton for F_2 ratio denominator
grv_version	92	Pion PDF: 92 (GRV92) or 99 (GRV99)

Run control and usability

Key	Default	Description
scan_mode	all	x, theta, kbin, or all
precision	custom	fast, normal, publication, custom
inclusive_mode	false	Remove all kinematic cuts (fully inclusive)
verbose	true	Print per-point progress to stdout
angle_min_deg	—	Alias: lower polar angle [deg] → **cosph_max**
angle_max_deg	—	Alias: upper polar angle [deg] → **cosph_min**

x-scan settings

The x-scan integrates $\theta_h \in$ [**theta_min**, **theta_max**] and $|\vec{k}| \in$ [**k_min**, **k_max**] (with **nk** steps) at each output x.

Key	Default	Description
x_min	0.02	Minimum Bjorken-x
x_max	0.30	Maximum Bjorken-x
nx	100	Number of output x points
n_theta_integration	100	Internal θ_h steps

theta-scan settings

Key	Default	Description
theta_min	0.0	Lower θ_h bound [deg]
theta_max	100.0	Upper θ_h bound [deg]
nth	100	Number of output θ_h points
n_x_integration	100	Internal x steps
x_min_th	10^{-4}	Internal x lower bound
x_max_th	0.6	Internal x upper bound
k_min_th	10^{-4}	Internal $\|\vec{k}\|$ lower bound [GeV]
k_max_th	0.5	Internal $\|\vec{k}\|$ upper bound [GeV]

kbin-scan settings

Key	Default	Description
k_min	0.050	Lower $\lvert\vec{k}\rvert$ bin bound [GeV]
k_max	0.100	Upper $\lvert\vec{k}\rvert$ bin bound [GeV]
cosph_min	0.342	Min $\cos\phi_k$ ($= \cos 70°$)
cosph_max	0.866	Max $\cos\phi_k$ ($= \cos 30°$)
ny	100	Steps in y integration
nkt	50000	Steps in k_T integration

Performance note: **nkt=50000** matches the Fortran exactly. In v1.3 a kbin run with **nx=50**, **ny=100**, **nkt=50000** takes ≈ 1 s on a laptop, so there is little reason to lower it. With **nkt=5000** the result changes by $\lesssim 0.2\%$, rising to $\sim 1\%$ only next to the kinematic end point where $F_2^{(\pi N)} \to 0$.

Output settings

Key	Default	Description
output_mode	`all`	`ascii`, `root`, `all`, `none`
output_name	`PionCloud`	File stem for output files

Output files

With **output_name = MyRun**:

File	Content
`MyRun_params.txt`	Full run parameters + first moments
`MyRun_xscan.dat`	x, Q^2, $F_2^{(\pi N)}$, running moment [, F_2^n, ratio]
`MyRun_thetascan.dat`	θ_h, $F_2^{(\pi N)}$, running moment
`MyRun.root`	TGraphs, TH1s, TCanvases, TNamed parameters

ROOT objects

Object names encode the channel and scan mode using the convention `<channel>_<mode>`:

- channel: `piN`, `piN_neutral`, `rhoN`, `piDelta`
- mode: `xscan`, `theta`, `kbin`

Name	Type	Description
g_<pfx>	TGraph	$F_2^{(\text{channel})}(x)$
g_moment_<pfx>	TGraph	Running $\int F_2\,dx$
h_<pfx>	TH1D	Same, as histogram
c_<pfx>	TCanvas	Log-scale plot
g_F2neutron	TGraph	$F_2^n(x)$ from CTEQ6M (always this name)
g_ratio_<pfx>	TGraph	$F_2^{(\text{channel})}/F_2^n$
h_ratio_<pfx>	TH1D	Ratio as histogram
c_ratio_<pfx>	TCanvas	Two-pad: curves (top) + ratio (bottom)
g_thetascan	TGraph	$F_2^{(\pi N)}(\theta_h)$
g_moment_theta	TGraph	Running $\int F_2\,d\theta$
h_thetascan	TH1D	Same, as histogram
c_thetascan	TCanvas	Angular distribution plot
RunParams	TNamed	Full parameter summary string

What's new in v1.3

Nucleon selector

The reference nucleon for the ratio $F_2^{(\pi N)}/F_2^{\text{nucleon}}$ can now be chosen:

```
nucleon = neutron    # default: F2n via charge symmetry (matches Fortran)
nucleon = proton     # F2p directly from CTEQ6
```

GRV99 pion PDF

An updated 1999 pion PDF parametrisation is available alongside the default GRV92:

```
grv_version = 92    # default: GRV92, matches original Fortran
grv_version = 99    # GRV99 update from f2_pi_sub.f90
```

Precision preset

Sets **ny** and **nkt** together for the kbin scan:

```
precision = fast          # ny=20,  nkT=500   -- quick checks
precision = normal        # ny=50,  nkT=2000  -- development
precision = publication   # ny=100, nkT=50000 -- matches Fortran
precision = custom        # use ny/nkT directly
```

With **precision = custom** (the default when **precision** is not given) the **ny**/**nkt** values of the input file, or the built-in defaults **ny=100**, **nkt=50000**, are used.

Inclusive mode shortcut

Removes all kinematic cuts, giving the fully inclusive $F_2^{(\pi N)}(x)$ *of the selected charge state* without setting four parameters by hand. Multiply by 3 (neutral) or 3/2 (charge_exchange) to obtain the violet "fully integrated" curve of the published figures, which sums both charge states:

```
inclusive_mode = true
```

Equivalent to setting **cosph_min=0**, **cosph_max=1**, **k_min=0.001**, **k_max=10.0**.

Verbosity control

```
verbose = false    # suppress all stdout (for library/generator embedding)
```

Angular cut in degrees

Aliases for **cosph_min**/**cosph_max**:

```
angle_min_deg = 30.0    # lower polar angle -> cosph_max = cos(30) = 0.866
angle_max_deg = 70.0    # upper polar angle -> cosph_min = cos(70) = 0.342
```

CTEQ6 table directory

Allows running from an arbitrary working directory:

```
p.cteq6_table_dir = "/path/to/cteq6_tables";
// Calculator chdir()s there before CTEQ6::init(), then restores cwd.
```

Single-point eval() for generator use

```
Calculator calc(p);
double f2pi = calc.eval(x, Q2);   // Q2 <= 0: computed from E,x,theta_e
```

Returns $F_2^{(\pi N)}(x, Q^2)$ for one point. Always silent regardless of `p.verbose`. Suitable for event-by-event generator calls.

C interface for Python/Fortran

```
#include "pioncloud_c.h"
pioncloud_init_default();                    // default BoNuS config
double f2pi = pioncloud_eval_f2pi(x, Q2);
double f2n  = pioncloud_eval_f2n (x, Q2);
double rat  = pioncloud_eval_ratio(x, Q2);
```

Python usage via `ctypes`:

```
import ctypes
lib = ctypes.CDLL("./libPionCloudLib.so")
lib.pioncloud_init_default.restype = None
lib.pioncloud_eval_f2pi.restype    = ctypes.c_double
lib.pioncloud_eval_f2pi.argtypes   = [ctypes.c_double, ctypes.c_double]
lib.pioncloud_init_default()
f2pi = lib.pioncloud_eval_f2pi(0.1, 2.0)
```

See `include/pioncloud_c.h` for full Fortran interface documentation.

Thread safety warning: CTEQ6 uses Fortran `COMMON` blocks and is not thread-safe. Do not call from multiple threads simultaneously.

Worked examples: the published TDIS figures

The Fortran codes ported in PionCloud were written by T. Hobbs for the pion-cloud calculations of the JLab TDIS proposal PR12-15-006 [1] (approved as E12-15-006; an earlier version is PR12-14-010 [2]), and several of the same curves appear in Ref. [3]. This section gives the input file for each published figure and overlays the PionCloud v1.3 result on the original curves.

How the comparison was made. The proposal figures are vector graphics (xmgrace). Each curve was extracted exactly from the PDF (no hand digitisation) by converting the page to SVG and calibrating on the axis frame and major tick marks. The extracted curves, the scripts and the input generator are in `docs/validation/` (see its `README.md`):

```
source /path/to/root/bin/thisroot.sh            # PyROOT + numpy needed
docs/validation/run_compare.sh build/PionCloud /tmp/pc_validation
# -> /tmp/pc_validation/plots/fig*.pdf, summary.txt (PionCloud/published ratios)
```

In every plot below, the **thick translucent** lines are the published curves and the **thin** lines of the same colour and style are PionCloud. The lower panel shows the ratio PionCloud/published.

Common settings. All figures use $E = 11$ GeV, $\theta_e = 35°$, GRV92 pion PDFs, CTEQ6M for $F_2^{p,n}$, the s-dependent exponential form factor (**`form_factor = exp_s`**) with $\Lambda = 1.56$ GeV and $\Lambda_{\pi\Delta} = 1.39$ GeV:

```
form_factor    = exp_s
lambda         = 1.56      # pi-N and rho-N vertices (HSS central value)
lambda_delta   = 1.39      # pi-Delta vertex (used automatically for pi_delta)
beam_energy    = 11.0
electron_angle = 35.0      # as in 3Var_x.f / 3Var_theta.f -- also for kbin
```

Ready-made inputs are in `input_files/` (e.g. `fig47_proposal/`, `fig37_proposal/`); the complete set used here is generated by `docs/validation/make_runs.sh`.

Fig. 47: angular distribution (theta scan)

Fig. 47 of [1] shows $F_2^{(\pi p)}(\theta_{p'}; \Delta x, \Delta|\vec{k}|)$ for neutral exchange $p \to \pi^0 p$ and $p \to \rho^0 p$. Left panel: $\Delta x = [0, 0.6]$, $\Delta|\vec{k}| = [0, 500]$ MeV (the theta-scan defaults); right panel: $\Delta x = [0.05, 0.6]$, $\Delta|\vec{k}| = [60, 250]$ MeV.

```
scan_mode    = theta
contribution = pion          # rho for the blue dashed curve
dissociation = neutral       # p -> pi0 p
theta_min = 0.0
theta_max = 90.0
nth       = 90
n_x_integration = 200
nk        = 200
# right panel only:
x_min_th  = 0.05             # Delta_x = [0.05, 0.6]
x_max_th  = 0.6
k_min_th  = 0.060            # Delta_k = [60, 250] MeV
k_max_th  = 0.250
```

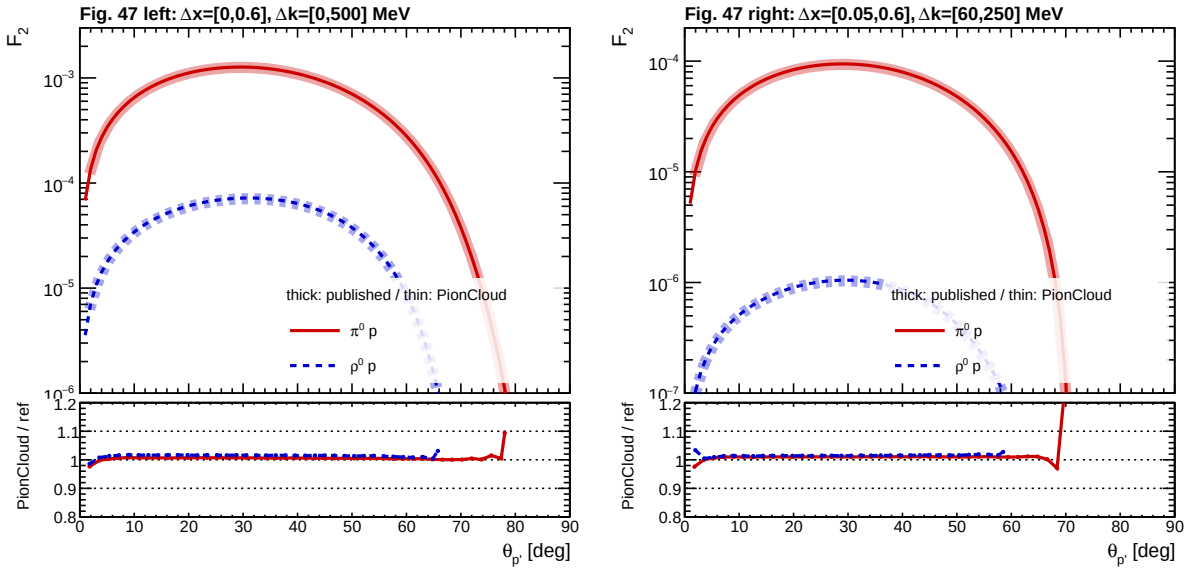


Figure 1: Fig. 47 of [1] (thick) and PionCloud theta scan (thin): $\pi^0 p$ (red) and $\rho^0 p$ (blue, dashed). Agreement is $\approx 1\%$ over the whole angular range.

Fig. 46: x dependence, full acceptance (x scan)

Fig. 46 of [1]: $F_2^{(Mp)}(x; \Delta|\vec{k}|, \Delta\theta_{p'})$ for $p \to \pi^0 p$ and $p \to \rho^0 p$, integrated over $\Delta|\vec{k}| = [0, 500]$ MeV and all angles. This is the `3Var_x.f` calculation:

```
scan_mode    = x
contribution = pion          # rho for the blue dashed curve
dissociation = neutral
x_min = 0.005
x_max = 0.4
nx    = 80
theta_min = 0.0              # all angles
theta_max = 180.0
n_theta_integration = 200
k_min = 0.0001               # Delta_k = [0, 500] MeV
k_max = 0.5
nk    = 200
```

Fig. 5: proton target, four $|\vec{k}|$ bins (kbin scan)

Fig. 5 of [1] (identical to Fig. 2, right, of [3]): $p \to \pi^0 p$ in the bins $\Delta|\vec{k}| = [60, 100]$ (black), $[100, 200]$ (red dashed), $[200, 300]$ (green dash-dotted) and $[300, 400]$ MeV (blue), compared with the total $F_2^{(\pi p)}(x)$ (violet dashed) and F_2^p (orange). The figure header says $\Delta\theta_{p'} = [30°, 180°]$; using 30°–70° instead gives the same result to $< 0.5\%$ (Sec. 7.6).

```
scan_mode    = kbin
contribution = pion
dissociation = neutral       # p -> pi0 p
nucleon      = proton        # F2p for the orange curve / ratio
x_min = 0.05
x_max = 0.30
nx    = 50
k_min = 0.060                # one run per bin:
k_max = 0.100                # 060-100, 100-200, 200-300, 300-400 MeV
```

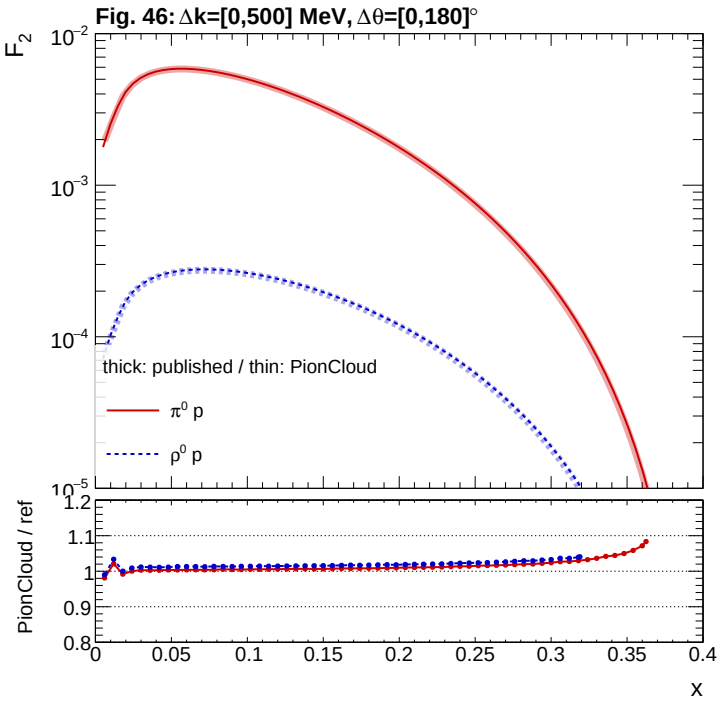


Figure 2: Fig. 46 of [1] (thick) and PionCloud x scan (thin).

```
cosph_min = 0.342            # theta_p' < 70 deg (cos 70)
cosph_max = 0.866            # theta_p' > 30 deg (cos 30)
ny    = 100
nkt   = 5000
# violet curve: replace k_min/k_max/cosph_* by  inclusive_mode = true
#               and multiply F2piK by 3 (sum of the two charge states)
```

Fig. 6: neutron, πN, ρN, $\pi\Delta$ channels (kbin scan)

Fig. 6 of [1]: charge exchange ($n \to \pi^- p$, $\rho^- p$ and $\pi^0\Delta^0 + \pi^-\Delta^+$) with $\Delta\theta_h = [30°, 180°]$. The momentum window is not given in the caption; it is reproduced with $\Delta|\vec{k}| = [60, 250]$ MeV, the full proposed RTPC acceptance.

```
scan_mode    = kbin
contribution = pion          # black;  rho: red dashed;  pi_delta: green
dissociation = charge_exchange
nucleon      = neutron
x_min = 0.05
x_max = 0.20
nx    = 50
k_min = 0.060
k_max = 0.250
cosph_min = 0.0              # 30-180 deg (cos(phi_k) > 0 always)
cosph_max = 0.866
ny    = 100
nkt   = 5000
# violet curve: inclusive_mode = true, multiplied by 3/2
```

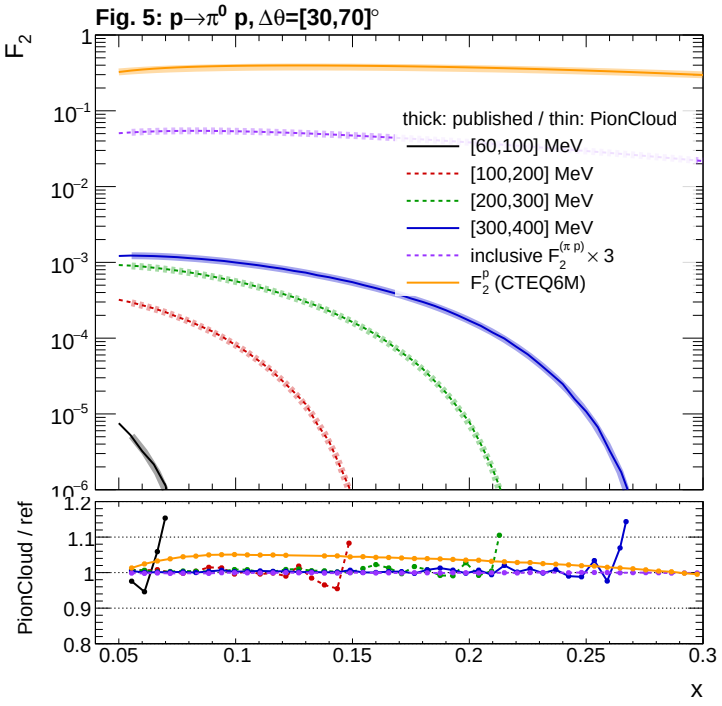


Figure 3: Fig. 5 of [1] (thick) and PionCloud kbin scan (thin). The inclusive PionCloud curve has been multiplied by 3. The large ratio excursions sit at the kinematic end points, where the curves fall steeply to zero.

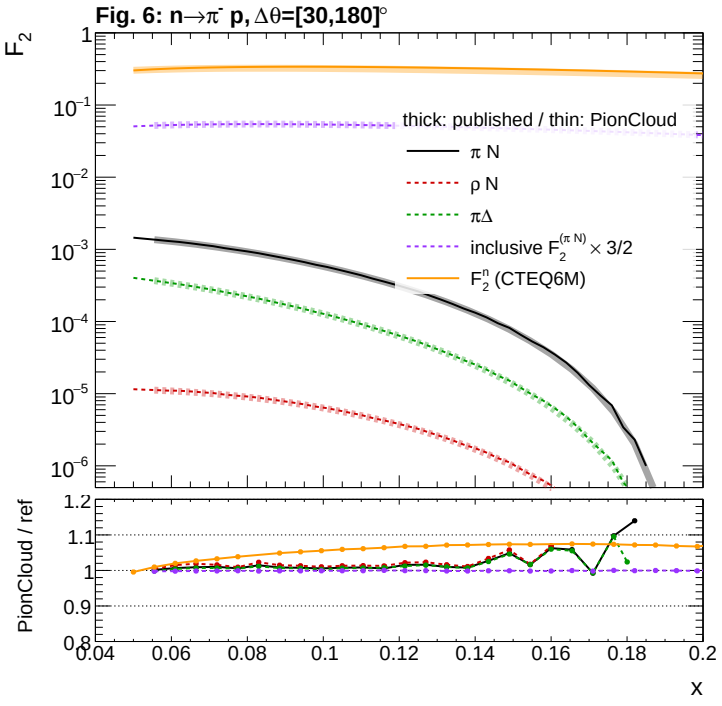


Figure 4: Fig. 6 of [1] (thick) and PionCloud (thin). The inclusive PionCloud curve has been multiplied by 3/2.

Fig. 37: neutron, $\Delta|\vec{k}| = [250, 400]$ MeV (kbin scan)

Fig. 37 of [1] (identical to Fig. 2, left, of [3]): same channels as Fig. 6, but with $\Delta|\vec{k}| = [250, 400]$ MeV and $\Delta\theta_p = [30°, 70°]$. These are the inputs in `input_files/fig37_proposal/`:

```
scan_mode    = kbin
contribution = pion          # rho / pi_delta for the other channels
dissociation = charge_exchange
nucleon      = neutron
x_min = 0.05
x_max = 0.30
nx    = 50
k_min = 0.250
k_max = 0.400
cosph_min = 0.342
cosph_max = 0.866
ny    = 100
nkt   = 5000
```

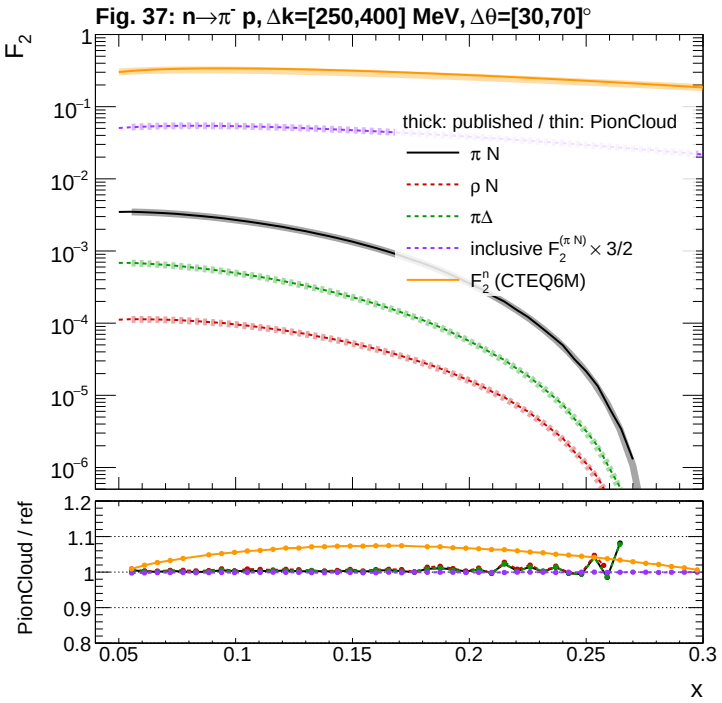


Figure 5: Fig. 37 of [1] (thick) and PionCloud (thin). The inclusive PionCloud curve has been multiplied by 3/2. The projected data points of the original figure are not shown.

Summary of the comparison

Figure	Curve	median dev.	max |dev.| (bulk)
47 left / right	$\pi^0 p$	+0.6% / +1.0%	2.4%
47 left / right	$\rho^0 p$	+1.5%	3.4%
46	$\pi^0 p$ / $\rho^0 p$	+0.8% / +1.7%	3.6% / 4.1%
5	$\|\vec{k}\|$ bins 100–400 MeV	+0.2–0.5%	3.5%
6	πN, ρN, $\pi\Delta$	+0.7–1.6%	7%
37	πN, ρN, $\pi\Delta$	+0.3–0.6%	2.8%
5, 6, 37	inclusive × isospin	−0.1%	0.3%
5, 6, 37	F_2^p, F_2^n (CTEQ6M)	+3.5–6.8%	7.5%

"Bulk" means points where the published curve is within a factor 100 of its maximum; closer to the kinematic end points both curves fall steeply to zero and the ratio is not meaningful. The largest bulk deviations (Fig. 6) occur at $x \approx 0.18$, just before the end point.

Lessons for reproducing the published curves:

- **Electron angle.** Use $\theta_e = 35°$ in all modes. With 12° the tagged curves come out 3–11% higher at $x \gtrsim 0.15$.
- **Charge states.** One run is one charge state. The "fully integrated" violet curves are the sum over charge states: 3× neutral, or $\frac{3}{2}$× charge exchange (agreement 0.1%).
- **$\pi\Delta$ cut-off.** The $\pi\Delta$ channel uses **`lambda_delta`**. Changing **`lambda`** does not affect it.
- **Angular window.** $\cos\phi_k > 0$ always, so $[30°, 180°]$ is entered as **`cosph_min = 0`**. For $x \geq 0.05$, $|\vec{k}| \leq 400$ MeV the 70° cut changes the result by $< 0.5\%$.
- **$F_2^{p,n}$.** The CTEQ6M F_2 used for the ratio lies 3–7% above the orange curves of the proposal. This affects only the reference curves and the ratio, not $F_2^{(\pi N)}$; the PDF set or order used for the published curves is not documented (open item).
- **Not reproduced here.** Fig. 35 of [1] (ratios with projected data, a ROOT raster image) and Fig. 45 (the $|\vec{k}|$ dependence; there is no $|\vec{k}|$-scan mode yet).

Class and API reference

RunParams (PhysicsParams.hh)

The central configuration struct. All fields have sensible defaults. Can be constructed and filled entirely in C++ without an input file.

```
#include "PhysicsParams.hh"
PionCloud::RunParams p;
p.flag       = PionCloud::FLAG_PION;
p.typ        = PionCloud::FF_EXP_S;
p.L          = 1.56;
p.dis        = PionCloud::DIS_CHARGE_EXCHANGE;
p.E          = 11.0;
p.theta_e    = 12.0;
p.kmin       = 0.060;
p.kmax       = 0.250;
p.cosph_min  = 0.342;
p.cosph_max  = 0.866;
p.ny         = 100;
p.nkT        = 5000;
```

```
p.outputMode = PionCloud::OUT_NONE;  // no file I/O
```

Key enumerations:

Enum	Values	Key
FormFactorType	FF_MONOPOLE..FF_PAULI_VILLAR	**form_factor**
ContributionFlag	FLAG_PION, FLAG_RHO, FLAG_PI_DELTA	**contribution**
DisChannel	DIS_CHARGE_EXCHANGE, DIS_NEUTRAL	**dissociation**
OutputMode	OUT_NONE, OUT_ASCII, OUT_ROOT, OUT_ALL	**output_mode**

Calculator (Calculator.hh)

The integration engine. No I/O dependencies.

```
#include "Calculator.hh"
PionCloud::Calculator calc(p);

PionCloud::ScanResults res;
calc.runXScan(res);         // x-scan (3Var_x.f)
calc.runThetaScan(res);     // theta-scan (3Var_theta.f)
calc.runKBinScan(res);      // kbin-scan (f2pi_sub.f90)

// Or run the mode selected by p.scanMode:
PionCloud::ScanResults res2 = calc.runAll();
```

Public methods:

Method	Description
`runXScan(out)`	x-scan; fills `out.xScan`
`runThetaScan(out)`	theta-scan; fills `out.thetaScan`
`runKBinScan(out)`	kbin-scan; fills `out.xScan`
`runAll()`	runs modes selected by `p_.scanMode`; returns `ScanResults`

Private static helpers:

Method	Description
`kMag(kT, y)`	3D recoil proton momentum magnitude from (k_T, y)
`cosPhi(kT, y)`	Proton polar angle $\cos\phi_k$ from (k_T, y)

ScanResults and data structures (Results.hh)

Plain structs with no ROOT or file-system dependencies:

```
struct XScanPoint {
    double x;           // Bjorken-x
    double Q2;          // Q^2 at this x [GeV^2]
    double F2piK;       // F2^pi(x) integrated over phase space
    double F2piMoment;  // running integral
    double F2n;         // F2^n from CTEQ6 (0 if unavailable)
    double ratio;       // F2piK / F2n
};

```

```cpp
struct ThetaScanPoint {
    double theta_deg;   // hadron production angle [degrees]
    double F2piK;       // F2^pi(theta_h)
    double F2piMoment;  // running integral
};

struct ScanResults {
    std::vector<XScanPoint>     xScan;
    std::vector<ThetaScanPoint> thetaScan;
    double F2pi0_x     = 0.0;  // first moment (Simpson)
    double F2pi0_theta = 0.0;
};
```

InputReader (InputReader.hh)

Parses INI-style input files. Optional — `RunParams` can be set directly in C++ without using this class.

```cpp
#include "InputReader.hh"
PionCloud::InputReader reader;
PionCloud::RunParams p = reader.read("myrun.in");

if (reader.hasErrors()) {
    reader.printDiagnostics();
    return 1;
}

// Generate a fully documented template file:
PionCloud::InputReader::writeTemplate("myrun.in");
```

OutputWriter (OutputWriter.hh)

Writes ASCII and/or ROOT output. Controlled by `p.outputMode`.

```cpp
#include "OutputWriter.hh"
PionCloud::OutputWriter writer(p, "MyRun");
writer.write(results);
```

CTEQ6 (CTEQ6.hh)

C++ wrapper for the CTEQ6 Fortran PDF library. Handles all float/double casts internally.

```cpp
#include "CTEQ6.hh"
PionCloud::CTEQ6::init();                       // call once
double f2n = PionCloud::CTEQ6::F2neutron(x, Q2);
double f2p = PionCloud::CTEQ6::F2proton (x, Q2);
double xfu = PionCloud::CTEQ6::xfx(1, x, Q);   // u-quark
```

Available functions:

Function	Description
`init(set)`	Initialise PDF tables (default: CTEQ6M, iset=1)
`xfx(iparton,x,Q)`	$x\,f(x,Q)$ for parton code `iparton`
`F2neutron(x,Q2)`	F_2^n via charge symmetry
`F2proton(x,Q2)`	F_2^p directly from CTEQ6

Parton codes: 0=gluon, 1=u, 2=d, 3=s, 4=c, 5=b; negative values for antiquarks.

Physics functions (SplittingFunctions.hh, GRV.hh, Kinematics.hh)

All inline, header-only, no dependencies:

Function	Description
`fypiN(y,kT,L,typ,dis)`	πN splitting function (TOPT)
`f_rhoN(y,kT,L,typ,dis)`	ρN splitting function
`f_RhoDel(y,kT,L,typ,dis)`	$\rho\Delta$ splitting function
`fypiD(y,kT,Ld,typ,dis)`	$\pi\Delta$ splitting function (SU(4))
`GRV92(xi,Q2,xV,xS)`	GRV92 LO pion PDFs
`F2piGRV(xi,Q2)`	$(5/9)(xV + 2xS)$: electromagnetic combination
`calcQ2(E,x,theta_e)`	Q^2 from beam kinematics
`simpsonWeight(i,n)`	Composite Simpson weight for index i of n

Using PionCloud as a library

PionCloud is built as a static library `libPionCloudLib.a`. Any C++ program can embed it:

```
# In your CMakeLists.txt:
add_subdirectory(PionCloud)
target_link_libraries(MyProgram PRIVATE PionCloudLib)
```

```
#include "PhysicsParams.hh"
#include "Calculator.hh"
#include "Results.hh"

using namespace PionCloud;

RunParams p;
p.flag       = FLAG_PION;
p.typ        = FF_EXP_S;
p.L          = 1.56;
p.dis        = DIS_CHARGE_EXCHANGE;
p.E          = 11.0;  p.theta_e = 12.0;
p.kmin       = 0.060; p.kmax    = 0.250;
p.cosph_min  = 0.342; p.cosph_max = 0.866;
p.ny  = 100;  p.nkT = 5000;
p.outputMode = OUT_NONE;   // no files written
p.scanMode   = "kbin";

Calculator calc(p);
ScanResults res;
calc.runKBinScan(res);

for (auto &pt : res.xScan)
```

```
24     printf("x=%.4f  Q2=%.4f  F2piK=%.6e  F2n=%.6e  ratio=%.6e\n",
25            pt.x, pt.Q2, pt.F2piK, pt.F2n, pt.ratio);
```

For event-by-event use, `Calculator::eval(x, Q2)` returns a single point silently, `p.verbose = false` suppresses all output, and `p.cteq6_table_dir` sets where the CTEQ6 tables are read from (see "What's new in v1.3").

Current limitation (see `TODO.md`): CTEQ6 is not thread-safe (Fortran `COMMON` blocks); do not evaluate from several threads at once.

CLI reference

```
./PionCloud [--input FILE] [OPTIONS]

--input FILE              Read parameters from INI file
--write-template FILE     Write documented template and exit

--mode x|theta|kbin|all   Scan mode
--output ascii|root|all|none  Output format
--out STEM                Output file stem

# Physics
--flag 0|1|2              Contribution (pion/rho/pi_delta)
--dis  0|1                Dissociation channel
--typ  0..6               Form factor type
--L    GeV                Lambda (pi/rho-N)
--Ld   GeV                Lambda (Delta)
--E    GeV                Beam energy
--theta_e DEG             Electron scattering angle

# x-scan
--xmin --xmax --nx --nth-int

# theta-scan
--thmin --thmax --nth --nx-int
--xmin-th --xmax-th --kmin-th --kmax-th

# kbin-scan / |k| window
--kmin --kmax --nk
--ny --nkt --cosph-min --cosph-max
```

Project layout

```
PionCloud/
|---- CMakeLists.txt
|---- README.md
|---- TODO.md
|---- input_files/           # ready-to-use inputs for published figures
|     |---- fig47_proposal/    # Fig.47 of PR12-15-006 (theta scan)
|     |---- fig37_proposal/    # Fig.37 (piN, rhoN, piDelta, 250-400 MeV)
|     |---- fig35_proposal/    # Fig.35 (50 MeV |k| bins, n and p)
|     |---- fig2_hobbs/        # Fig.2 of arXiv:1411.2216
|     `---- normal_proposal/   # piN, rhoN, piDelta, 60-250 MeV
|---- CTEQ6/
|     |---- Cteq6Pdf-2007.f    # Fortran PDF library (v6.52)
|     |---- cteq6m.tbl         # iset=1 NLO MSbar (default)
```

```
|     |---- cteq6d.tbl / cteq6l.tbl / cteq6l1.tbl
|     `---- README
|---- include/
|     |---- PhysicsParams.hh   # RunParams, enums
|     |---- SplittingFunctions.hh  # fypiN, f_rhoN, f_RhoDel, fypiD
|     |---- GRV.hh             # GRV92 pion PDFs
|     |---- Kinematics.hh      # Q2, phase space, Simpson weights
|     |---- Results.hh         # XScanPoint, ThetaScanPoint, ScanResults
|     |---- Calculator.hh      # main integration engine
|     |---- CTEQ6.hh           # extern C bridge to Fortran
|     |---- InputReader.hh     # INI file parser
|     `---- OutputWriter.hh    # ASCII + ROOT writer
|---- src/
|     |---- Calculator.cc
|     |---- InputReader.cc
|     |---- OutputWriter.cc
|     `---- main.cc
|---- tests/
|     |---- CMakeLists.txt
|     `---- test_all.cc        # 153 tests, 0 failures
`---- docs/
    |---- PionCloud_manual.tex   # this document
    |---- figures/               # comparison plots used in this manual
    `---- validation/            # extracted published curves + scripts
```

Known issues and TODO

The complete, current list is maintained in `TODO.md`. Main open items:

1. **Sum of channels**: a combined run giving $\pi N + \rho N + \pi\Delta$ (and the sum over charge states).
2. **Thread safety**: CTEQ6 Fortran `COMMON` blocks.
3. **Automated regression tests** against the published curves of Sec. 7 (x and theta scans in particular).
4. **$|\vec{k}|$-scan and t-scan modes**: $F_2^{(\pi N)}$ vs $|\vec{k}|$ (Fig. 45 of [1]) and vs Mandelstam $-t$, binning the (y, k_T) integrand by $t = -k_T^2 - m_N^2 y^2/(1-y)$.
5. **$F_2^{p,n}$ normalisation**: CTEQ6M is 3–7% above the F_2 curves of the published figures (Sec. 7.6).

References

[1] A. Camsonne *et al.*, "Measurement of Tagged Deep Inelastic Scattering (TDIS)", JLab proposal PR12-15-006 (PAC43, 2015), approved as E12-15-006; https://www.jlab.org/exp_prog/proposals/15/PR12-15-006.pdf.

[2] A. Camsonne *et al.*, "Measurement of Tagged Deep Inelastic Scattering (TDIS)", JLab proposal PR12-14-010 (PAC42, 2014); https://www.jlab.org/exp_prog/proposals/14/PR12-14-010.pdf.

[3] T. J. Hobbs, "Phenomenological implications of the nucleon's meson cloud", Few-Body Syst. **56**, 363 (2015), arXiv:1411.2216 [hep-ph].

[4] H. Holtmann, A. Szczurek and J. Speth, Nucl. Phys. A **596**, 631 (1996).

[5] M. Glück, E. Reya and A. Vogt, Z. Phys. C **53**, 651 (1992).

[6] J. Pumplin *et al.*, JHEP **0207**, 012 (2002).